Deliverable 2 - INF201

Submission deadline: 30th Oct 2026

Hand-in information:

- This is a group submission, so one person submits on behalf of the pair/group
- Submit the Jupyter notebook (ipynb) *and* a pdf of the same notebooks
  - Should contain code, results/plots and discussion in markdown cells
- Also submit the output files generated by your code
- Include a link to your GitHub repository and a log of the commits
  - To generate the log, paste this into the terminal after your last commit: `git log --graph --pretty=format:"%h - %an, %ar : %s" > git_log.txt`
  - The repository should contain a `requirements.txt` with information on your virtual environment
  - Note: Remember to create a new virtual environment for each deliverable
- Include a discussion of GenAI use, for example a summary of the chat log as you have done earlier
  - This can be a separate file or be included in the Jupyter notebook
- Make a short (5 mins) video discussing your code and results
  - Note that you both have to contribute to the video
  - Either by recording it together
  - Or recording parts separately and editing it together

Scenario - Campus Coffee Shop Sales Pipeline

You have been hired as a Python developer for the Campus Coffee Shop Chain. The chain operates several coffee stands across different campus buildings. This is an international campus, so the stands accept payment in several currencies.

Each cash register saves transaction logs as text files. However, the legacy system has several issues:

- Mixed Character Encodings: Older registers write text files using ISO-8859-1 (Latin-1) encoding, while newer registers write files using UTF-8.
- System Noise: Registers intersperse valid sales lines with printer and network debug log lines.
- Nested Folder Hierarchy: Log files are stored across nested subdirectories organized by year, store, and register ID.
- Data Errors: System glitches record occasional extreme price outliers (e.g., $999.00), and prices in NOK are by error registered using EUR-scale values (e.g., 4.50 NOK instead of 45.00 NOK).

Your task is to set up Python code that discovers all log files, cleans the transaction data, calculates price metrics and generates summary reports.
Note: In this assignment, you write the functions you will use in tasks 1-4, and then put everything together in task 5. You should therefore read through the entire assignment text before you start, to fully understand what you will use each function for.

Overview of inputs and outputs for this assignment

Inputs (in `data/` folder)

- You generate the input data on you own laptop by running the script `generate_data_for_d2.py`
- `data/store_config.json` : A nested JSON file containing store metadata.
- `data/2026/...` : A folder hierarchy containing subdirectories with .log files in mixed character encodings.

Outputs (files your code should generate)

- `cleaned_transactions.csv` : A CSV table containing all parsed, outlier-cleaned, and currency-corrected transactions.
- `hourly_revenue.json` : A JSON report containing total revenue per hour per item.
- `status_pivot_table.csv` : A CSV file containing a pivot table of total transaction revenue grouped by item across status categories.
- `item_refund_ratios.csv` : A CSV table containing refund ratios for each item type.

Task 1: File ingestion and regular expressions

1. Write a robust file reader function:
   - Use `pathlib.Path` to handle file paths.
   - The function should be able to handle both `UTF-8` and `ISO-8859-1/Latin-1` encoding. You may assume that these are the only two encodings the files can have. Note that the function should handle the encoding, you may not hard-code which encoding each file has.
   - Return a tuple containing two strings: `(file_text_content, encoding_used)` .
2. Write a log parser function that uses regular expressions. Log files contain valid sales lines mixed with system noise.
   Valid line format:
   `[2026-10-15 08:30:00] [POS_01] item=Latte price=4.50 EUR (SUCCESS)`
   Noise lines to ignore:
   `--- PRINTER PAPER LOW WARNING ---`
   `=== NETWORK SYNC OK ===`
   - Construct a regular expression (`re` module) to capture the 6 key fields from valid lines:
     - `timestamp` (string, format: `YYYY-MM-DD HH:MM:SS`)

        - `register_id` (string, e.g., `POS_01`)
        - `item` (string, e.g., `Latte`)
        - `price` (float, e.g., `4.50`)
        - `currency` (string, e.g., `EUR`, `NOK`, `USD`)
        - `status` (string: `SUCCESS`, `REFUND`, or `CANCELLED`)
    - Return a list of Python dictionaries, where each dictionary represents one valid transaction record.

Task 2: Recursive utility functions

1. Write a recursive directory search function that finds all log files in a nested directory and returns a list of `pathlib.Path` objects:
    - Convert the input path to a `pathlib.Path` object.
    - Iterate through items using `Path.iterdir()`.
    - If an item is a directory, recursively search inside it.
    - If an item is a file with a matching extension (`.log`), add its Path object to a list.
    - Constraint: Do NOT use `Path.glob()`, `Path.rglob()`, or `os.walk()`. You must implement explicit recursion.
2. Metadata about the different stores can be found in the `data/store_config.json` file. This file contains deeply nested dictionaries and lists (see example below). Write a recursive function that:
    - Traverses any combination of nested dictionaries and lists found in this file.
    - Returns a set (type hint: `set[str]`) of all register id strings where `"active": true`.

Example config.json:

```
{
  "company": "Campus Coffee",
  "regions": [
    {
      "region_name": "Main Campus",
      "stores": [
        {
          "name": "Library Cafe",
          "registers": [
            {"id": "POS_01", "active": true},
            {"id": "POS_02", "active": false}
          ]
        }
      ]
    }
  ]
}
```

Task 3: Functions for cleaning and analysing data using NumPy

1. Write a function for removing price outliers that:
    - Converts input price data to a 1D NumPy float array.
    - Detects hardware glitch outliers (999.00) where a value deviates from the mean by more than 3 standard deviations:

$$|x - \mu| > 3\sigma$$

    - Replaces detected outlier values with the median of the valid prices.
    - Returns the cleaned 1D array.
2. Write a function `compute_refund_ratio(sales_count: np.ndarray, refund_count: np.ndarray) -> np.ndarray` that:
    - Computes the element-wise refund ratio using NumPy array operations:

$$\text{Refund Ratio} = \frac{\text{Refund Count}}{\text{Sales Count} + 10^{-6}}$$

    - The addition of $\epsilon = 10^{-6}$ prevents zero-division errors.

Task 4: Data analysis using Pandas

1. Write a function that:
    - Converts the list of transaction dictionaries read from the log files into a `pandas.DataFrame`.
    - Converts the `timestamp` column to `datetime` and sets it as the DataFrame index.
    - Sorts the DataFrame chronologically by timestamp.
2. Write a function that has the DataFrame you just created as input, groups data by item and resamples transactions in 1-hour windows (1h), calculating total price (sum).
3. Write a function that has the DataFrame as input and creates a pivot table displaying total transaction revenue grouped by item across status categories (`SUCCESS`, `REFUND`, `CANCELLED`).
4. Write a function that has the DataFrame as input and calculates the refund ratio per item by calling the function you made in task 3.

Task 5: Create full workflow

Combine the functions you have made into a workflow that does the following:

1. Recursively find all log files and extract active register IDs.
2. Ingest text logs and parse records into a list of dictionaries.
3. Build the initial Pandas DataFrame.
4. Apply NumPy outlier cleaning to remove 999.00 price glitches.
5. Apply currency correction: multiply price by 10.0 for all transactions where `currency == "NOK"`.
6. Generate resampled reports, pivot tables and refund ratios.

7. Save all output files listed in the introduction (Scenario: Overview of inputs and outputs for this assignment).